量子 LLM 支援プログラミングにおける 失敗類型と介入カタログの実証研究

2023TC011 伊藤 悠真　　指導教員：横山 哲郎

1 はじめに

大規模言語モデル（LLM）の発展に伴いコード生成が進み，HumanEval［1］や MBPP［2］で性能が評価されている．一方，生成コードには構文・意味的誤りや，存在しない API を用いるハルシネーションが生じ得る．量子プログラミングは量子ビットの重ね合わせ・量子もつれと確率的な測定を扱うため，Python として実行できても期待する量子状態・測定分布を得られるとは限らない．段階的な正しさの検証が必要である．

本研究は，Qiskit コードの初期失敗に対する修正介入の救済率と救済対象の違いを明らかにし，失敗類型に応じた介入カタログの根拠を整理することを目的とする．

2 評価の枠組み

現行評価器では，L0 は構文・import を含むコード読込みと必須関数への適合，L1 は回路構築と評価処理の実行，L2 は指定入力での期待出力との一致，主 L3 は L0〜L2 と構造要件への適合とする．関数名・戻り値・出力形式などのインターフェース適合性と，量子的な出力一致を区別する．

構造要件は量子ビット数や，測定・barrier を除く命令数の範囲などである．主 L3 は限定された基準であり，全入力のアルゴリズム同値性を保証しない．測定分布は 8,192 shots の全変動距離，状態ベクトルは大域位相を整合した距離を用い，0.05 未満を一致とする．測定先は量子ビット順に戻す．救済率は「初期失敗から最終主 L3 成功となった組数／初期主 L3 失敗組数」とし，失敗類型は介入前で固定する．

3 関連研究・準備

Self-Refine［3］は自己フィードバックと反復修正，Self-Debugging［4］はコード説明や実行結果を用いる．推論課題では，外部フィードバックのない自己修正の限界も報告されている［5］．Qiskit は Python で量子回路を構築する基盤であり，Qiskit HumanEval［6］はそのコード生成を評価する．本研究では独自の 10 タスクに修正条件を実装し，先行研究の完全再現とは位置づけない．予備実験から対象・介入を拡張したが，旧 L2 評価の 120 件は主結果に合算しない．

4 実験方法

5 モデル・10 タスク・seed 0〜19 の 1,000 初期組を対象に，5 条件，計 5,000 条件記録を分析した．モデル略号は Q4：qwen3.5:4b, G4：gemma4:e4b-it-qat, Q9：qwen3.5:9b，C7：qwen2.5-coder:7b，G3：gemma3:4b である．

タスクは Bell，GHZ，Deutsch − Jozsa（constant/balanced），Bernstein − Vazirani，Grover，QFT，IQFT，QPE，Ansatz とした．temperature は 0.7，1 要求の生成上限は 1,024 tokens である．

B は初期生成，R は自己見直し，F はエラー等の実行結果提示，D は説明・原因分析を促す自己デバッグ，P は生成前注意事項である．R/F/D は B と同じ初期コード・判定から最大 3 回修正し，成功時に停止する．P は別プロンプトによる独立生成である．

検証済み保存記録と部分追試 293 ケースを反映した到達可能な経路を集計した［7］．全件を新規生成・再実行した結果ではない．今回の執筆で追加実験は行っていない．

5 結果

初期成功は 455 組，失敗は 545 組であった．救済は R が 96/545 組（17.6%），F が 183/545 組（33.6%），D が 180/545 組（33.0%）であった．全体率は初期失敗 545 組を分母とする合算値である．

表 1　モデル別の救済件数（分母は初期失敗）

モデル	分母	R	F	D
G3	190	5	1	1
G4	62	30	48	38
C7	65	15	23	23
Q4	162	31	78	90
Q9	66	15	33	28

表 1 は各モデル 200 初期組のうち失敗した組を分母とする．G4 と Q4 では F と D の大小が入れ替わる．

表 2　初期失敗類型別の救済件数

初期失敗	分母	R	F	D
回路構築	126	32	76	68
import	271	24	58	72
IF 不適合	11	1	0	1
量子ビット数	3	2	0	0
構文	68	21	25	21
unknown	1	0	0	0
出力不一致	65	16	24	18

表 2 の import 失敗は F 58/271 組（21.4%），D 72/271 組（26.6%），回路構築失敗は F 76/126 組（60.3%），D 68/126 組（54.0%）であった．IF はインターフェースを表す．保存分類 unknown の 1 組は生成コード側の失敗であり，API 接続障害ではない．少数分母の率は参考値とする．

表 3　F と D の救済対象（初期失敗 545 組）

共通	F のみ	D のみ	双方未救済
140	43	40	322

F と D の全体の救済数は 3 組差だが，表 3 の片側のみ救済した 83 組では対象が異なる．import 失敗は F のみ 8 組・D のみ 22 組，回路構築失敗は 19 組・11 組であった．83 組すべての両条件 166 履歴・398 修正遷移を静的に確認した．複数箇所が同時に変わるため，変更内容から救済原因は分離できない．少なくとも一方が救済した 223/545 組（40.9%）は事後的な集合であり，実行前の選択方策の成功率ではない．

P では B の初期失敗に対応する成功が 124/545 組（22.8%），初期成功に対応する失敗が 106/455 組（23.3%）で，最終成功は 473/1,000 組だった．独立生成の結果を同一コードの修正による救済と同じ作用とは解釈しない．

共有初期生成を除き，未救済例への消費も含めた追加入力・出力 tokens ／救済は R 18,852.6，F 9,333.8，D 10,976.6 であった．保存記録と部分追試の再構成値であり，全件新実験の実測費用ではない．3 回目は F で 28/390 組（7.2%），D で 33/398 組（8.3%）を追加救済した．2 回打切りでは 3 回で救済できた組の 15.3%，18.3%をそれぞれ失う．これは同一履歴の打切り比較である．

6 考察

F と D の救済率が近くても救済対象は一致せず，初期失敗類型・モデルによって大小が変わった．介入カタログには全体率だけでなく，適用対象，失敗分母，追加消費を併記する必要がある．ただし，失敗類型はモデル・タスクと関連しており，この結果だけで介入選択規則を確定できない．

R/F/D は最大 3 回の修正機会を持つ一方，B/P は独立した 1 回生成であり，要求回数・投入情報が異なる．同じ要求回数の独立再生成対照もなく，因果効果を断定しない．成功時停止，初期コード・タスク・seed の依存，元実験の診断文差もあり，5,000 条件を独立標本として統計的有意差や一般的優劣を主張しない．主 L3 の入力範囲，有限 shots，生成長上限も限界である．

7 おわりに

初期失敗 545 組に対し，F は 33.6%，D は 33.0%を救済したが，片側のみ救済した 83 組の内訳は異なった．この違いを介入カタログの根拠として整理した．今後は介入選択・停止方策を事前固定し，要求回数をそろえた独立再生成対照と別 seed・タスクによる検証を行う．

参考文献

［1］M. Chen et al.：Evaluating Large Language Models Trained on Code，arXiv:2107.03374（2021）.

［2］J. Austin et al.：Program Synthesis with Large Language Models，arXiv:2108.07732（2021）.

［3］A. Madaan et al.：Self-Refine: Iterative Refinement with Self-Feedback，arXiv:2303.17651（2023）.

［4］X. Chen et al.：Teaching Large Language Models to Self-Debug，arXiv:2304.05128（2023）.

［5］J. Huang et al.：Large Language Models Cannot Self-Correct Reasoning Yet，arXiv:2310.01798（2023）.

［6］S. Vishwakarma et al.：Qiskit HumanEval: An Evaluation Benchmark For Quantum Code Generative Models，arXiv:2406.14712（2024）.

［7］現行到達経路の固定集計・保存設定・出典 hash（2026-10-16 版）．本原稿 README に出典と検証記録を示す．